

OS LabTask 9 - Fixed Size Memory Partitioning

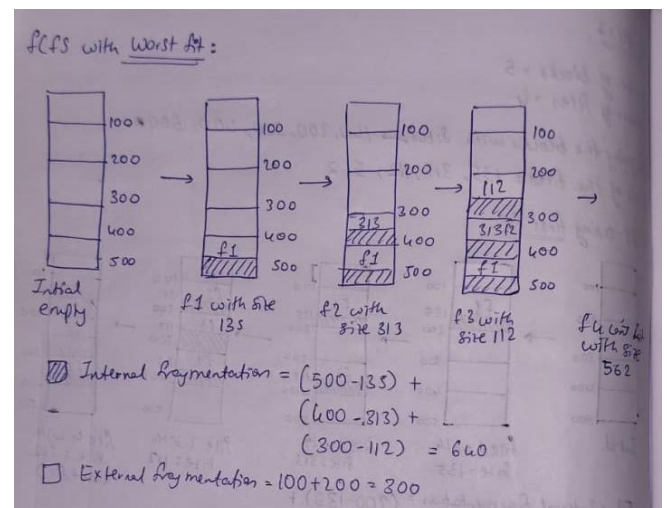
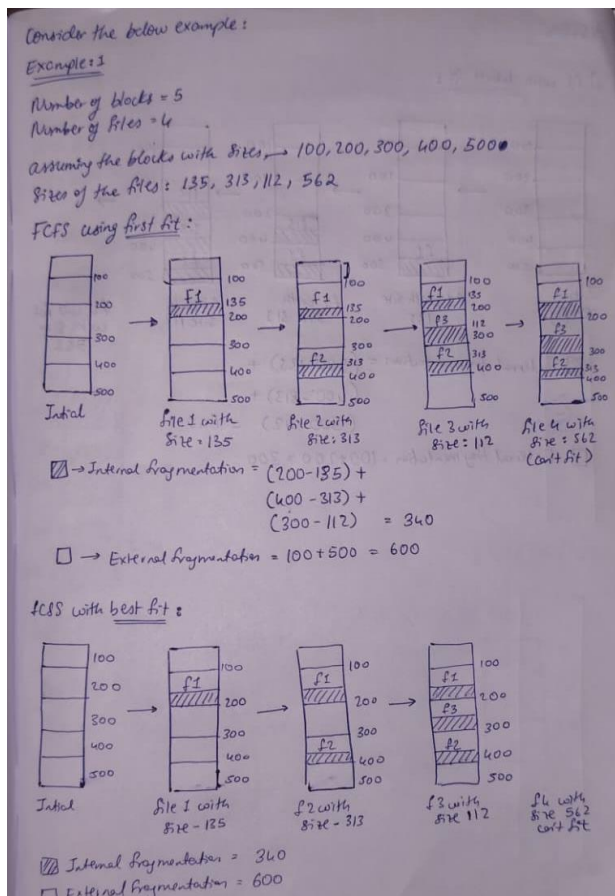
Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita

Aim 1: To implement first fit, best fit and worst fit algorithm for fixed sized memory management algorithm assuming **FCFS process** scheduling algorithm.

Example 1:

- Enter the number of blocks: 5
- Enter the number of files: 4
- Enter the size of the blocks:
 - Block 1: 100
 - Block 2: 200
 - Block 3: 300
 - Block 4: 400
 - Block 5: 500
- Enter the size of the files:
 - File 1: 135
 - File 2: 313
 - File 3: 112
 - File 4: 562

Manual Implementation for Example 1:



Code implementing Fixed Size Memory Partitioning using FCFS

a. First Fit function

```
#include <stdio.h>
#include <stdlib.h>

void firstFit(int blockSize[], int blocks, int fileSize[], int files) {
    int allocated[blocks];
    int intFragment[blocks];
    for(int i=0; i<blocks; i++) {
        allocated[i] = 0;
        intFragment[i] = 0;
    }
    printf("\nFirst Fit Allocation:\n");
    printf("File No\tFile Size\tBlock No\tBlock Size\tIFragment\n");
    for(int i=0; i<files; i++) {
        int found = 0;
        for(int j=0; j<blocks; j++) {
            if(!allocated[j] && blockSize[j] >= fileSize[i]) {
                allocated[j] = 1;
                intFragment[j] = blockSize[j] - fileSize[i];
                printf("%d\t%d\t%d\t%d\t%d\n", i+1, fileSize[i], j+1, blockSize[j], intFragment[j]);
                found = 1;
                break;
            }
        }
        if(!found) {
            printf("%d\t%d\t\t--\t\t--\t\t--\n", i+1, fileSize[i]);
        }
    }
    int totalInternal = 0, totalExternal = 0;
    for(int i=0; i<blocks; i++) {
        if(allocated[i]) totalInternal += intFragment[i];
        else totalExternal += blockSize[i];
    }
    printf("Total Internal Fragmentation: %d\n", totalInternal);
    printf("Total External Fragmentation: %d\n", totalExternal);
}
```

b. Best Fit function

```

void bestFit(int blockSize[], int blocks, int fileSize[], int files) {
    int allocated[blocks];
    int intFragment[blocks];
    for(int i=0; i<blocks; i++) {
        allocated[i] = 0;
        intFragment[i] = 0;
    }
    printf("\nBest Fit Allocation:\n");
    printf("File No\tFile Size\tBlock No\tBlock Size\tIFragment\n");
    for(int i=0; i<files; i++) {
        int bestIdx = -1, minFrag = 1000000;
        for(int j=0; j<blocks; j++) {
            if(!allocated[j] && blockSize[j] >= fileSize[i]) {
                int frag = blockSize[j] - fileSize[i];
                if(frag < minFrag) {
                    minFrag = frag;
                    bestIdx = j;
                }
            }
        }
        if(bestIdx != -1) {
            allocated[bestIdx] = 1;
            intFragment[bestIdx] = blockSize[bestIdx] - fileSize[i];
            printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, fileSize[i], bestIdx+1, blockSize[bestIdx], intFragment[bestIdx]);
        } else {
            printf("%d\t%d\t%d\t--\t--\t--\t--\n", i+1, fileSize[i]);
        }
    }
    int totalInternal = 0, totalExternal = 0;
    for(int i=0; i<blocks; i++) {
        if(allocated[i]) totalInternal += intFragment[i];
        else totalExternal += blockSize[i];
    }
    printf("Total Internal Fragmentation: %d\n", totalInternal);
    printf("Total External Fragmentation: %d\n", totalExternal);
}

```

c. Worst Fit function

```
void worstFit(int blockSize[], int blocks, int fileSize[], int files) {
    int allocated[blocks];
    int intFragment[blocks];
    for(int i=0; i<blocks; i++) {
        allocated[i] = 0;
        intFragment[i] = 0;
    }
    printf("\nWorst Fit Allocation:\n");
    printf("File No\tFile Size\tBlock No\tBlock Size\tIFragment\n");
    for(int i=0; i<files; i++) {
        int worstIdx = -1, maxFrag = -1;
        for(int j=0; j<blocks; j++) {
            if(!allocated[j] && blockSize[j] >= fileSize[i]) {
                int frag = blockSize[j] - fileSize[i];
                if(frag > maxFrag) {
                    maxFrag = frag;
                    worstIdx = j;
                }
            }
        }
        if(worstIdx != -1) {
            allocated[worstIdx] = 1;
            intFragment[worstIdx] = blockSize[worstIdx] - fileSize[i];
            printf("%d\t%d\t%d\t%d\t%d\t%d\n", i+1, fileSize[i], worstIdx+1, blockSize[worstIdx], intFragment[worstIdx]);
        } else {
            printf("%d\t%d\t%d\t--\t--\t--\n", i+1, fileSize[i]);
        }
    }
    int totalInternal = 0, totalExternal = 0;
    for(int i=0; i<blocks; i++) {
        if(allocated[i]) totalInternal += intFragment[i];
        else totalExternal += blockSize[i];
    }
    printf("Total Internal Fragmentation: %d\n", totalInternal);
    printf("Total External Fragmentation: %d\n", totalExternal);
}
```

d. Driver code

```
int main() {
    int blocks, files;
    printf("Enter the Number of blocks: ");
    scanf("%d", &blocks);
    int *blockSize = (int*)malloc(blocks * sizeof(int));
    printf("Enter the block sizes: ");
    for(int i=0; i<blocks; i++) {
        scanf("%d", &blockSize[i]);
    }
    printf("Enter the Number of files: ");
    scanf("%d", &files);
    int *fileSize = (int*)malloc(files * sizeof(int));
    printf("Enter the file sizes: ");
    for(int i=0; i<files; i++) {
        scanf("%d", &fileSize[i]);
    }
    int *blockSizeFF = (int*)malloc(blocks * sizeof(int));
    int *blockSizeBF = (int*)malloc(blocks * sizeof(int));
    int *blockSizeWF = (int*)malloc(blocks * sizeof(int));
    for(int i=0; i<blocks; i++) {
        blockSizeFF[i] = blockSize[i];
        blockSizeBF[i] = blockSize[i];
        blockSizeWF[i] = blockSize[i];
    }
    firstFit(blockSizeFF, blocks, fileSize, files);
    bestFit(blockSizeBF, blocks, fileSize, files);
    worstFit(blockSizeWF, blocks, fileSize, files);
    free(blockSize);
    free(fileSize);
    free(blockSizeFF);
    free(blockSizeBF);
    free(blockSizeWF);
    return 0;
}
```

Example 1 Output:

```
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ gcc fcfsFixedSize.c -o example1.exe
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ ./example1.exe
Enter the Number of blocks: 5
Enter the block sizes: 100 200 300 400 500
Enter the Number of files: 4
Enter the file sizes: 135 313 112 562

First Fit Allocation:
File No File Size      Block No      Block Size      IFragment
1         135           2           200             65
2         313           4           400             87
3         112           3           300            188
4         562           --           --              --
Total Internal Fragmentation: 340
Total External Fragmentation: 600

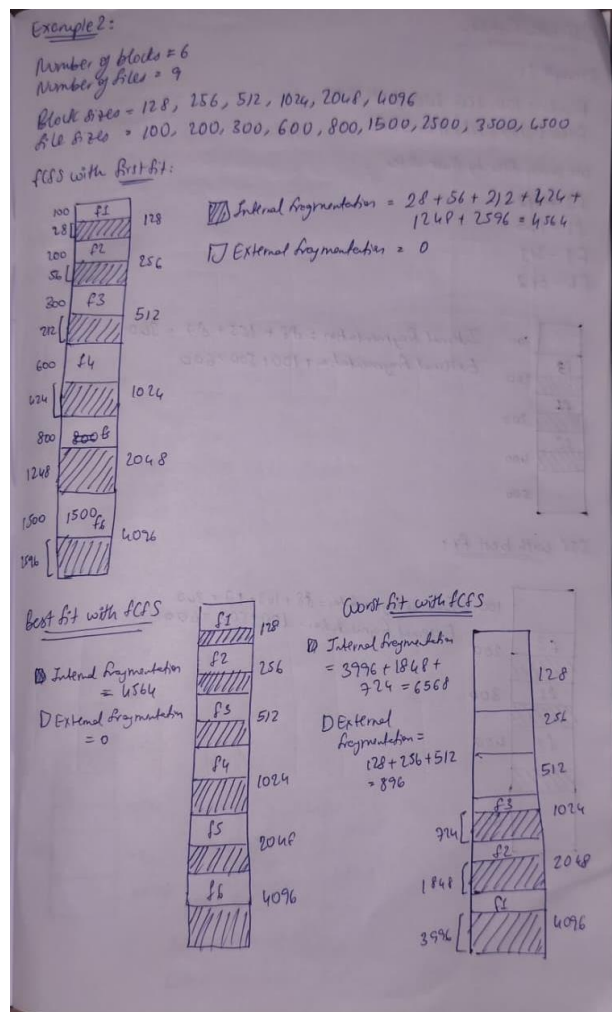
Best Fit Allocation:
File No File Size      Block No      Block Size      IFragment
1         135           2           200             65
2         313           4           400             87
3         112           3           300            188
4         562           --           --              --
Total Internal Fragmentation: 340
Total External Fragmentation: 600

Worst Fit Allocation:
File No File Size      Block No      Block Size      IFragment
1         135           5           500            365
2         313           4           400             87
3         112           3           300            188
4         562           --           --              --
Total Internal Fragmentation: 640
Total External Fragmentation: 300
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$
```

Example 2: Own Scenario

1. Enter the number of blocks: 6
2. Enter the number of files: 9
3. Enter the size of the blocks:
 - a. Block 1: 128
 - b. Block 2: 256
 - c. Block 3: 512
 - d. Block 4: 1024
 - e. Block 5: 2048
 - f. Block 6: 4096
4. Enter the size of the files:
 - a. File 1: 100
 - b. File 2: 200
 - c. File 3: 300
 - d. File 4: 600
 - e. File 5: 800
 - f. File 6: 1500
 - g. File 7: 2500
 - h. File 8: 3500
 - i. File 9: 4500

Manual Implementation for Example 2:



Example 2 Output:

```
Total External Fragmentation: 300
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ gcc fcfsFixedSize.c -o example2.exe
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ ./example2.exe
Enter the Number of blocks: 6
Enter the block sizes: 128 256 512 1024 2048 4096
Enter the Number of files: 9
Enter the file sizes: 100 200 300 600 800 1500 2500 3500 4500

First Fit Allocation:
File No File Size      Block No      Block Size      IFragment
1         100          1           128             28
2         200          2           256             56
3         300          3           512            212
4         600          4          1024            424
5         800          5          2048            1248
6        1500          6          4096            2596
7        2500          --           --              --
8        3500          --           --              --
9        4500          --           --              --
Total Internal Fragmentation: 4564
Total External Fragmentation: 0

Best Fit Allocation:
File No File Size      Block No      Block Size      IFragment
1         100          1           128             28
2         200          2           256             56
3         300          3           512            212
4         600          4          1024            424
5         800          5          2048            1248
6        1500          6          4096            2596
7        2500          --           --              --
8        3500          --           --              --
9        4500          --           --              --
Total Internal Fragmentation: 4564
Total External Fragmentation: 0

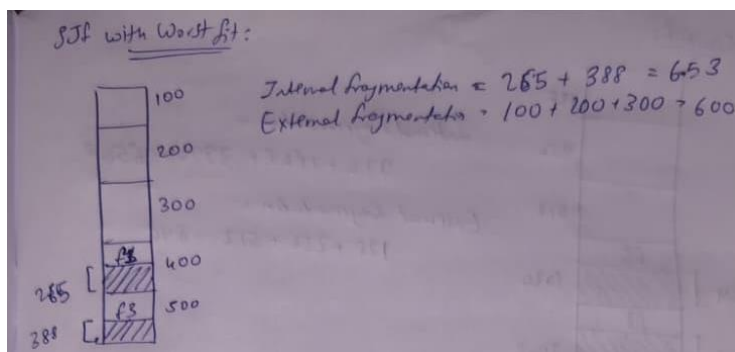
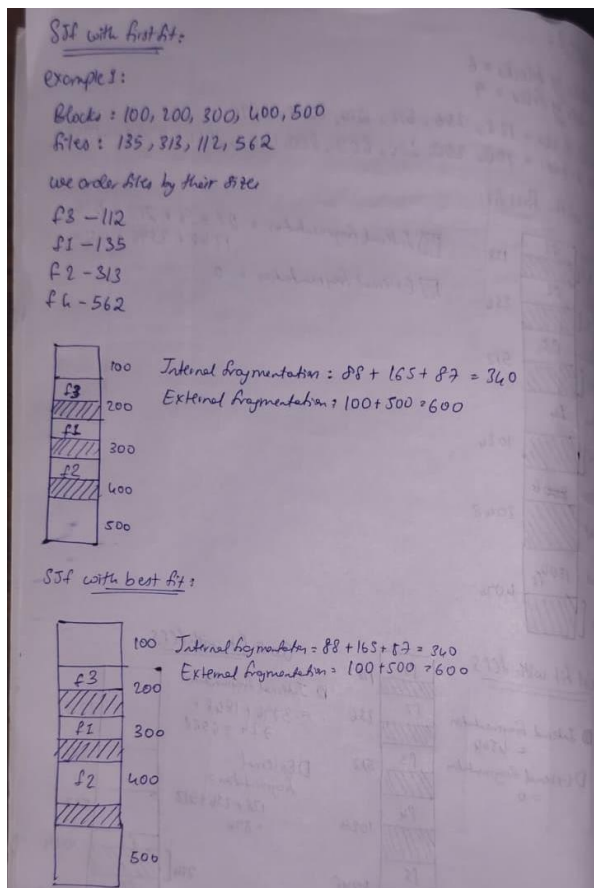
Worst Fit Allocation:
File No File Size      Block No      Block Size      IFragment
1         100          6          4096            3996
2         200          5          2048            1848
3         300          4          1024             724
4         600          --           --              --
5         800          --           --              --
6        1500          --           --              --
7        2500          --           --              --
8        3500          --           --              --
9        4500          --           --              --
Total Internal Fragmentation: 6568
Total External Fragmentation: 896
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$
```

Aim 2: To implement first fit, best fit and worst fit algorithm for fixed sized memory management algorithm assuming **SJF process** scheduling algorithm. Use the above two scenarios for verification.

Example 1:

- Enter the number of blocks: 5
- Enter the number of files: 4
- Enter the size of the blocks:-
 - Block 1: 100
 - Block 2: 200
 - Block 3: 300
 - Block 4: 400
 - Block 5: 500
- Enter the size of the files:-
 - File 1: 135
 - File 2: 313
 - File 3: 112
 - File 4: 562

Manual Implementation for Example 1:



Example 2: Own Scenario

1. Enter the number of blocks: 6
2. Enter the number of files: 9
3. Enter the size of the blocks:
 - a. Block 1: 128
 - b. Block 2: 256
 - c. Block 3: 512
 - d. Block 4: 1024
 - e. Block 5: 2048
 - f. Block 6: 4096

4. Enter the size of the files:

- File 1: 100
- File 2: 200
- File 3: 300
- File 4: 600
- File 5: 800
- File 6: 1500
- File 7: 2500
- File 8: 3500
- File 9: 4500

Manual Implementation for Example 2:

STF with FirstFit:

Example 2:

Blocks: 128, 256, 512, 1024, 2048, 4096

Files: 100, 200, 300, 600, 800, 1500, 2500, 3500, 4500

STF Order:

f1 - 100	f6 : 1500
f2 - 200	f7 : 2500
f3 - 300	f8 : 3500
f4 - 600	f9 : 4500
f5 - 800	

First fit

Internal fragmentation = 4564

External fragmentation = 0

Best fit

Internal fragmentation = 4564

External fragmentation = 0

Worst fit

Internal fragmentation = 6564

External fragmentation = 896

Code implementing Fixed Size Memory Partitioning using SJF

a. First fit function

```
#include <stdio.h>
#include <stdlib.h>

struct File {
    int size;
    int originalIndex;
};

int compareFiles(const void *a, const void *b) {
    struct File *fa = (struct File *)a;
    struct File *fb = (struct File *)b;
    return fa->size - fb->size;
}

void FF(struct File filesArr[], int files, int blockSize[], int blocks) {
    int allocated[blocks];
    int intFragment[blocks];
    for(int i=0; i<blocks; i++) {
        allocated[i] = 0;
        intFragment[i] = 0;
    }
    printf("\nFirst Fit Allocation (SJF):\n");
    printf("File No\tFile Size\tBlock No\tBlock Size\tIFragment\n");
    for(int i=0; i<files; i++) {
        int found = 0;
        for(int j=0; j<blocks; j++) {
            if(!allocated[j] && blockSize[j] >= filesArr[i].size) {
                allocated[j] = 1;
                intFragment[j] = blockSize[j] - filesArr[i].size;
                printf("%d\t%d\t%d\t%d\t%d\n", filesArr[i].originalIndex+1, filesArr[i].size, j+1, blockSize[j], intFragment[j]);
                found = 1;
                break;
            }
        }
        if(found) {
            printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n", filesArr[i].originalIndex+1, filesArr[i].size);
        }
    }
    int totalInternal = 0, totalExternal = 0;
    for(int i=0; i<blocks; i++) {
        if(allocated[i]) totalInternal += intFragment[i];
        else totalExternal += blockSize[i];
    }
    printf("Total Internal Fragmentation: %d\n", totalInternal);
    printf("Total External Fragmentation: %d\n", totalExternal);
}
```

b. Best fit function

```
void BF(struct File filesArr[], int files, int blockSize[], int blocks) {
    int allocated[blocks];
    int intFragment[blocks];
    for(int i=0; i<blocks; i++) {
        allocated[i] = 0;
        intFragment[i] = 0;
    }
    printf("\nBest Fit Allocation (SJF):\n");
    printf("File No\tFile Size\tBlock No\tBlock Size\tIFragment\n");
    for(int i=0; i<files; i++) {
        int bestIdx = -1, minFrag = 1000000;
        for(int j=0; j<blocks; j++) {
            if(!allocated[j] && blockSize[j] >= filesArr[i].size) {
                int frag = blockSize[j] - filesArr[i].size;
                if(frag < minFrag) {
                    minFrag = frag;
                    bestIdx = j;
                }
            }
        }
        if(bestIdx != -1) {
            allocated[bestIdx] = 1;
            intFragment[bestIdx] = blockSize[bestIdx] - filesArr[i].size;
            printf("%d\t%d\t%d\t%d\t%d\t%d\n", filesArr[i].originalIndex+1, filesArr[i].size, bestIdx+1, blockSize[bestIdx], intFragment[bestIdx]);
        } else {
            printf("%d\t%d\t%d\t%d\t-1\t-1\n", filesArr[i].originalIndex+1, filesArr[i].size);
        }
    }
    int totalInternal = 0, totalExternal = 0;
    for(int i=0; i<blocks; i++) {
        if(allocated[i]) totalInternal += intFragment[i];
        else totalExternal += blockSize[i];
    }
    printf("Total Internal Fragmentation: %d\n", totalInternal);
    printf("Total External Fragmentation: %d\n", totalExternal);
}
```

c. Worst fit function

```
void WF(struct File filesArr[], int files, int blockSize[], int blocks) {
    int allocated[blocks];
    int intFragment[blocks];
    for(int i=0; i<blocks; i++) {
        allocated[i] = 0;
        intFragment[i] = 0;
    }
    printf("\nWorst Fit Allocation (SJF):\n");
    printf("File No\tFile Size\tBlock No\tBlock Size\tIFragment\n");
    for(int i=0; i<files; i++) {
        int worstIdx = -1, maxFrag = -1;
        for(int j=0; j<blocks; j++) {
            if(!allocated[j] && blockSize[j] >= filesArr[i].size) {
                int frag = blockSize[j] - filesArr[i].size;
                if(frag > maxFrag) {
                    maxFrag = frag;
                    worstIdx = j;
                }
            }
        }
        if(worstIdx != -1) {
            allocated[worstIdx] = 1;
            intFragment[worstIdx] = blockSize[worstIdx] - filesArr[i].size;
            printf("%d\t%d\t%d\t%d\t%d\t%d\n", filesArr[i].originalIndex+1, filesArr[i].size, worstIdx+1, blockSize[worstIdx], intFragment[worstIdx]);
        } else {
            printf("%d\t%d\t%d\t%d\t-1\t-1\n", filesArr[i].originalIndex+1, filesArr[i].size);
        }
    }
    int totalInternal = 0, totalExternal = 0;
    for(int i=0; i<blocks; i++) {
        if(allocated[i]) totalInternal += intFragment[i];
        else totalExternal += blockSize[i];
    }
    printf("Total Internal Fragmentation: %d\n", totalInternal);
    printf("Total External Fragmentation: %d\n", totalExternal);
}
```


d. Driver code

```
int main() {
    int blocks, files;
    printf("Enter the Number of blocks: ");
    scanf("%d", &blocks);
    int *blockSize = (int*)malloc(blocks * sizeof(int));
    printf("Enter the block sizes: ");
    for(int i=0; i<blocks; i++) {
        scanf("%d", &blockSize[i]);
    }
    printf("Enter the Number of files: ");
    scanf("%d", &files);
    int *fileSize = (int*)malloc(files * sizeof(int));
    printf("Enter the file sizes: ");
    for(int i=0; i<files; i++) {
        scanf("%d", &fileSize[i]);
    }

    struct File *filesArr = (struct File*)malloc(files * sizeof(struct File));
    for(int i=0; i<files; i++) {
        filesArr[i].size = fileSize[i];
        filesArr[i].originalIndex = i;
    }
    qsort(filesArr, files, sizeof(struct File), compareFiles);
    int *blockSizeFF = (int*)malloc(blocks * sizeof(int));
    int *blockSizeBF = (int*)malloc(blocks * sizeof(int));
    int *blockSizeWF = (int*)malloc(blocks * sizeof(int));

    for(int i=0; i<blocks; i++) {
        blockSizeFF[i] = blockSize[i];
        blockSizeBF[i] = blockSize[i];
        blockSizeWF[i] = blockSize[i];
    }
    FF(filesArr, files, blockSizeFF, blocks);
    BF(filesArr, files, blockSizeBF, blocks);
    WF(filesArr, files, blockSizeWF, blocks);
    free(blockSize);
    free(fileSize);
    free(filesArr);
    free(blockSizeFF);
    free(blockSizeBF);
    free(blockSizeWF);
    return 0;
}
```

Example 1 Output:

```
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ vim sjfFixedSize.c
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ gcc sjfFixedSize.c -o eg1.exe
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ ./eg1.exe
Enter the Number of blocks: 5
Enter the block sizes: 100 200 300 400 500
Enter the Number of files: 4
Enter the file sizes: 135 313 112 562

First Fit Allocation (SJF):
File No File Size      Block No      Block Size      IFragment
3         112           2             200             88
1         135           3             300            165
2         313           4             400             87
4         562           --             --              --
Total Internal Fragmentation: 340
Total External Fragmentation: 600

Best Fit Allocation (SJF):
File No File Size      Block No      Block Size      IFragment
3         112           2             200             88
1         135           3             300            165
2         313           4             400             87
4         562           --             --              --
Total Internal Fragmentation: 340
Total External Fragmentation: 600

Worst Fit Allocation (SJF):
File No File Size      Block No      Block Size      IFragment
3         112           5             500            388
1         135           4             400            265
2         313           --             --              --
4         562           --             --              --
Total Internal Fragmentation: 653
Total External Fragmentation: 600
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$
```

Example 2 Output:

```
Total External Fragmentation: 800
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ gcc sjfFixedSize.c -o eg2.exe
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$ ./eg2.exe
Enter the Number of blocks: 6
Enter the block sizes: 128 256 512 1024 2048 4096
Enter the Number of files: 9
Enter the file sizes: 100 200 300 600 800 1500 2500 3500 4500

First Fit Allocation (SJF):
File No File Size      Block No      Block Size      IFragment
1         100           1          128           28
2         200           2          256           56
3         300           3          512          212
4         600           4         1024          424
5         800           5         2048         1248
6        1500           6         4096         2596
7        2500           --           --           --
8        3500           --           --           --
9        4500           --           --           --
Total Internal Fragmentation: 4564
Total External Fragmentation: 0

Best Fit Allocation (SJF):
File No File Size      Block No      Block Size      IFragment
1         100           1          128           28
2         200           2          256           56
3         300           3          512          212
4         600           4         1024          424
5         800           5         2048         1248
6        1500           6         4096         2596
7        2500           --           --           --
8        3500           --           --           --
9        4500           --           --           --
Total Internal Fragmentation: 4564
Total External Fragmentation: 0

Worst Fit Allocation (SJF):
File No File Size      Block No      Block Size      IFragment
1         100           6         4096         3996
2         200           5         2048         1848
3         300           4         1024          724
4         600           --           --           --
5         800           --           --           --
6        1500           --           --           --
7        2500           --           --           --
8        3500           --           --           --
9        4500           --           --           --
Total Internal Fragmentation: 6568
Total External Fragmentation: 896
bhaavya@bhaavya-VirtualBox:~/Desktop/25MCB1010$
```